ABAcore Animátor

Metodická príručka

Vytvorenie a nastavenie animátora

Vytvorenie animátora

Trieda Simulation obsahuje metódy *createAnimator*() a *setAnimator*(*IAnimator*) na vytvorenie alebo nastavenie animátora. Vytvorený animátor je možné vložiť do simulátora pred spustením simulácie, ale aj počas behu simulácie. Zavolaním metódy *createAnimator*() sa vytvorí základný animátor ABASIM-u.

```
// vytvorenie základného animátora
sim.createAnimator();
// alebo
sim.setAnimator(new Animator(sim));

// vráti či animátor je vytvorený
sim.animatorExists()
```

Pridanie do GUI

Na pridanie animátora do grafického rozhrania stačí z animátora získať *JComponent* pomocou *canvas*() a ten pridať na akékoľvek miesto do Swing používateľského rozhrania. Veľkosť animátora sa automaticky prispôsobuje veľkosti rodičovského prvku, do ktorého bol vložený. Prípadne je možné upraviť veľkosť, napríklad nastavením *setLayout* rodičovského prvku a veľkosti *setBounds* animátora.

```
JFrame frame = new JFrame();
frame.setBounds(0, 0, 1000, 800);

// pridanie animátora do GUI
frame.add(sim.animator().canvas());

frame.setVisible(true);
frame.setLayout(null);

// nastavenie veľkosti animátora
sim.animator().canvas().setBounds(0, 0, 500, 500);
```

V prípade C# verzie využíva animátor WPF grafické rozhranie a metóda *canvas*() vráti *FrameworkElement*. Ten je možné podobne vložiť do WPF grafického rozhrania, prípadne je možné WPF adaptér *ElementHost* (WindowsFormsIntegration.dll) a vložiť ho do grafického rozhrania WinForms.

```
// príklad použitia C# WPF
var window = new Window
{
     Width = 1000, Height = 800, Content = sim.Animator.Canvas
};
new System.Windows.Application().Run(window);

// pridanie do WinForms použitím ElementHost
WPFHost = new ElementHost();
WPFHost.Dock = DockStyle.Fill;
WPFHost.Child = sim.Animator.Canvas;
Controls.Add(WPFHost);
```

Nastavenie animátora

Animátor ponúka viacero možností na jeho prispôsobenie. Medzi tieto možnosti patrí nastavenie obrázku pozadia, farby pozadia a relatívnej pozície plátna spolu s priblížením. Rýchlosť animátora sa mení automaticky spolu so simuláciou, a preto ju nie je potrebné manuálne nastavovať. Trieda Animator ponúka aj viacero odvodených možností, ako napríklad nastavenie pozadia pomocou cesty k súboru (setBackgroundImagePath), nastavenie relatívnej pozície na pravý horný roh (setViewPosition) a nastavenie rozmerov zobrazenej plochy (setView).

```
// nastavenie pozadia
sim.animator().setBackgroundImage(Config.IMG_POZADIE);

// nastavenie relatívnej pozície
// (stred plátna bude bod 1000,1000 s priblížením 2)
// môže byť použité len ak animátor má nastavenú veľkosť
sim.animator().setViewPositionCenter(1000,1000,2)
```

Okrem tohto je možné nastaviť režim synchronizácie so simulačným modelom. Synchronizácia animácie a simulácie má dve varianty: predvolenú blokujúcu synchronizáciu, pri ktorej si jadrá navzájom odovzdávajú riadenie, a neblokujúci režim, v ktorom animačné jadro synchronizuje čas na základe pravidelných aktualizácií.

```
// zmena režimu synchronizácie true -> blokovaná, false -> neblokovaná
sim.animator().setSynchronizedTime(true);
```

Bočné menu

Animátor obsahuje aj grafické menu nachádzajúce sa na ľavej strane obrazovky. Obsahuje tlačidlá na priblíženie, oddialenie, spustenie nahrávania, otvorenie nahrávky a otvorenie nastavení.



Vymazanie animátora

Na vymazanie animátora stačí zavolať jednu z metód *removeAnimator()* alebo *setAnimator(null)* triedy Simulation, prípadne ešte odstrániť jeho komponent z GUI.

```
// príklad odstránenia animátora z GUI
frame.remove(sim.animator().canvas());

// odstránenie animátora zo simulácie
sim.removeAnimator();
```

Notifikácie o zmene animátora

Sledovanie vytvorenia a odstránenia animátora je možné vložením kódu, ktorý sa má vykonať pri zmene animátora, prostredníctvom lambda funkcií.

```
sim.onAnimatorWasCreated((oldAnimator, newAnimator) -> {
        System.out.println("Animator was created.");
});

sim.onAnimatorWasRemoved((oldAnimator) -> {
        System.out.println("Animator was removed.");
});
```

Použitie animátora

Animačné objekty

Úlohou animátora je zobrazovať stav simulácie. To sa deje za použitia animačných objektov. Počas behu simulácie vytvárame a rušíme animačné objekty, s ktorými manipulujeme, a tým animujeme simuláciu.
Animátor ABASIM-u používa tri základné animačné objekty predstavujúce: obrázok, text a tvar. Animačný objekt obrázka je reprezentovaný triedou *AnimImageItem*, animačný objekt textu triedou *AnimTextItem* a animačný objekt tvaru triedou *AnimShapeItem*. Všetky tieto triedy majú spoločného predka – triedu *AnimItem*. S animačnými objektmi sa dá pracovať kdekoľvek, ale odporúča sa ich používať v manažéroch, asistentoch a kontinuálnych asistentoch.

Vytváranie animačných objektov

```
//príklady vytvorenia animačných objektov

//obrázok
_animObject = new AnimImageItem(Config.IMG_CASNIK);
//tvar
_animObject = new AnimShapeItem(AnimShape.CIRCLE,Color.red,30);
//text
_animObject = new AnimTextItem("text",Color.yellow,_font);
```

Registrácia a získanie animačných objektov

Aby bol animačný objekt zobrazovaný, musí sa zaregistrovať do animátora. Zaregistrovanie animačného objektu prebieha zavolaním metódy *register()*. Po zaregistrovaní animačného objektu do animátora prechádza animačný objekt pod jeho správu a animátor sa stará o jeho aktualizácie a zobrazovanie. Pri pokuse o zaregistrovanie už registrovaného animačného objektu do animátora sa vypíše chyba. Pri registrácii je každému objektu pridelené unikátne ID, ktoré môže používateľ aj manuálne nastaviť alebo zmeniť. Po registrácii môže používateľ kdekoľvek využiť toto ID na získanie daného animačného objektu.

```
// zaregistrovanie animačného objektu _animObject
mySim.animator().register(_animObject);

// zaregistrovanie animačného objektu _animObject s unikátnym id
mySim.animator().register(int id, _animObject);
```

```
//registrácia v čase simTime
mySim.animator().registerLater(_animObject, double simTime);

//získanie animačného objektu podľa unikátneho id
AnimItem item = mySim.animator().getAnim(int id);
```

Prispôsobenie animačných objektov

V prípade použitia vstavaných animačných objektov má používateľ k dispozícii viaceré možnosti na ich prispôsobenie. K dispozícii sú 3 vstavané animačné objekty: obrázok, text a tvar.

- **Text** - V prípade textu sa dá nastaviť jeho text, font a veľkosť písma.

Java	C#
`item.setText("text");`	`item.Text = "text";`
`item.setFont(new Font("Arial", Font.PLAIN, 14);`	`item.Font = new Typeface("Arial");`
`item.setSize(20);`	`item.Size = 20;`
`item.setColor(Color.blue);`	`item.Color = Colors.Blue`

- **Obrázok** – Dá sa vytvoriť z už existujúceho obrázka alebo stačí nastaviť cestu k súboru obrázka. Používateľ si môže nastaviť aj vlastnú šírku a výšku obrázka. Taktiež je možné obrázok horizontálne aj vertikálne prevrátiť.

Java	C#
`item.setImage("/cesta/obrazok.png");` `item.setImage(ImageIO.read(...));`	`Item.Image = new BitmapImage(...);` `item.ImagePath = "/cesta/obrazok.png";`

`item.setWidth(10);`
`item.setHeight(10);`
`item.setImageSize(10, 10);`
	`item.Width = 20;`
	`item.Height = 20;`

`item.flipImage();`	`item.FlipImage();`

- **Tvar** - V prípade tvaru môže používateľ využiť jeden z predpripravených tvarov: štvorec, kruh, trojuholník, hviezda, kríž, päťuholník alebo šesťuholník. Taktiež však môže použiť aj vlastný vytvorený tvar. Okrem toho môže nastaviť jeho farbu, výplň a rozmery.

Java	C#
`item.setAnimShape(AnimShape.CIRCLE);`	`item.AnimShape = AnimShape.CIRCLE;`
`item.setShape(new Rectangle(0,0,10,10));`	`item.Shape = Geometry.Empty;`

`item.setWidth(10);`
`item.setHeight(10);`
`item.setSize(10); // w & h = 10`
	`item.Width = 10;`
	`item.Height = 10;`
	`item.Size = 10; // w & h = 10`

`item.setColor(Color.blue);`	`item.Color = Colors.Black;`
`item.setFill(true);`	`item.Fill = true;`

Každý animačný objekt obsahuje aj všeobecné možnosti prispôsobenia. Medzi tieto patrí napríklad nastavenie relatívnej pozície vykresľovania. Vďaka tomu môže používateľ jednoducho nastaviť, či pozícia (X, Y) predstavuje stred, alebo napríklad niektorý roh objektu. Je možné nastaviť aj vrstvu (Z-index), na ktorej sa bude animačný objekt nachádzať. Animačný objekt na vyššej vrstve bude prekrývať objekty na nižších vrstvách.
Animačné objekty obsahujú aj možnosť nastavenia automatickej rotácie, aby sa objekt otáčal podľa smeru svojho pohybu. Taktiež je možné nastaviť, či je objekt viditeľný, prípadne či je viditeľný aj bez aktívnej animácie (*AlwaysVisible*).
Každému animačnému objektu sa dá nastaviť aj štítok s textom, ktorý sa zobrazí vedľa animačného objektu. Taktiež je možné nastaviť pomocný text, ktorý sa zobrazí ako štítok pri podržaní kurzora myši nad daným objektom.
Poslednou možnosťou je nastavenie sledovania animačného objektu. Ak je objekt sledovaný, bude sa nachádzať v strede obrazovky a pri jeho pohybe sa plátno bude automaticky posúvať.

Java	C#
`item.setzIndex(10)`	`item.ZIndex = 10;`
`item.setPositionAlignment(PositionAlignment.CENTER);`	`item.PositionAlignment = PositionAlignment.CENTER;`
`item.setFollowed(true)`	`item.setFollowed(true);`
`item.setVisible(true);` `item.setAlwaysVisible(true);`	`item.Visible = true;` `item.AlwaysVisible = true;`
`item.setAutoRotate(true);` `item.setInitialDirection(180);`	`item.AutoRotate = true;` `item.InitialDirection = 180;`

`item.setLabel("text");`	`item.Label = "text";`
`item.setToolTip("text");`	`item.ToolTip = "text";`

Animácia objektov

Registrované animačné objekty je možné zobraziť a manipulovať s nimi rôznymi spôsobmi. Na tento účel sa používajú metódy *setPosition*, *startAnim*, *moveTo*, *move* a *wait*.

```
// nastavenie začiatočnej pozície objektu
_animObject.setPosition(double x, double y);

// nastavenie pozície v určitom čase
_animObject.setPosition(double startTime, double duration,Point2D p);

// animácia pohybu objektu po ceste path v určitom čase
_animObject.startAnim(double startTime, double duration, Point2D[] path);

// animácia pohybu objektu od posledného bodu do nového bodu (x, y) v určitom
    čase
_animObject.moveTo(double startTime, double duration, double x, double y);

// animácia pohybu objektu do smeru xDiff, yDiff od posledného bodu v určitom
    čase
// (xDiff = 50 -> pohyb doprava o 50 pixelov)
_animObject.move(double startTime, double duration, int xDiff, int yDiff);

// nastavenie pozície na určitý čas
_animObject.wait(double startTime, double duration, double x, double y);

*V prípade C# verzie sa namiesto Point2D využíva PointF, a metódy začínajú
    veľkým písmenom napr. MoveTo().
```

Odobratie animačného objektu z animátora

Ak niektorá entita v simulácii, predstavovaná animačným objektom, ukončila svoju činnosť (napr. zákazník odišiel z reštaurácie), odporúča sa odobrať animačný objekt zobrazujúci túto entitu z animátora.

Odobratie animačného objektu sa vykoná buď zavolaním metódy *remove(AnimItem)* na animátore, alebo zavolaním metódy *remove()* priamo na animačnom objekte. Animačný objekt je možné neskôr znova zaregistrovať.

```
// odstránenie z animátora
_animObject.remove();

// alebo
mySim().animator().remove(_animObject);
mySim().animator().remove(int id);

// odstránenie z animátora s oneskorením
_animObject.remove(mySim().currentTime() + 100);
```

Používanie príkazov

Objekty *Command* slúžia na vykonávanie používateľom definovaných akcií spojených s animátorom alebo konkrétnym animačným objektom. Pozostávajú zo simulačného času a metód *check, execute a isFinished*, ktoré si používateľ môže jednoducho prekryť a definovať, kedy a aká akcia sa vykoná. Na vykonanie týchto príkazov je potrebné ich pridať do animátora alebo príslušného animačného objektu.

K dispozícii sú aj odvodené triedy *LambdaCommand* a *LambdaAnimatorCommand*, cez ktoré je možné použiť aj lambda výrazy.

```
// pridanie vlastného príkazu
item.addCommand(myCommand);
animator.addCommand(myCommand);

// ako anonymná trieda (len pre Java verziu)
item.addCommand(new Command(10)
{
    @Override
    public void execute(double simTime, AnimItem item) {
        item.setToolTip("text"); //v čase simTime sa nastaví tooltip
string
    }
});

// ako lambda výraz
item.addCommand(10, (simTime, animItem) -> {
     animItem.setToolTip(text");
});
// C#
item.AddCommand(10, (simTime, animItem) => {
    animItem.ToolTip = "text";
});
```

Pomocné nástroje

Pre uľahčenie práce s animačnými objektami a prípadné hľadanie chýb pri vytváraní obsahuje animátor aj rôzne pomocné nástroje. Používateľ si napríklad môže jednoducho zobraziť pomocnú mriežku. Mriežka sa automaticky prispôsobuje mierke zväčšenia a posunu, pričom sa dynamicky upravuje rozostup čiar tak, aby boli vždy čitateľné. Každá čiara je umiestnená v pravidelných intervaloch v súradnicovom systéme, pričom hodnoty sú zobrazované na dolnom a ľavom okraji obrazovky. V tomto režime sa zobrazuje aj aktuálna pozícia myši.
Taktiež sa dajú zobraziť cesty animačných objektov, vrátane každého bodu a času animácií. Zobrazené môžu byť cesty všetkých animačných objektov, prípadne len jedného pri podržaní kurzora myši nad objektom.
Animátor obsahuje aj viacero statických vlajok v triede *Flags*, ktoré sa dajú kedykoľvek zapnúť pre prípadné zobrazenie výpisov internej činnosti niektorej časti animátora.
Posledné pomocné zobrazenie slúži na zobrazenie aktuálneho stavu animátora s výpismi ako celkový počet animácií, interný simulačný čas a podobne.

```
// zobrazenie mriežky
a.drawGrid(true);
// zobrazenie všetkých ciest
a.setDrawAnimsDebug(true);
// zobrazenie cesty animačného objektu podržaním myši
a.setDrawAnimsDebugOnHover(true);

// príklad nastavenia statickej vlajky na zobrazenie FPS a štatistík
animátora
Flags.DEBUG_ANIM = true;
```

Animácia čakania v rade

Pre uľahčenie vytvárania animácií čakania animačných objektov v rade sme vytvorili pomocný objekt animovaného radu (*AnimQueue*). Toto môže byť často vyžadované napríklad pri tvorbe simulácií obslužných systémov. Vďaka tomu stačí používateľovi vytvoriť objekt radu a nastaviť mu začiatočnú polohu, koncovú polohu a trvanie prechodu animačného objektu cez rad. Po vložení animačného objektu do tohto radu sa tento objekt automaticky presunie na posledné miesto v rade. Pri vybratí prvého prvku z radu sa všetky čakajúce animácie automaticky posunú o jedno miesto dopredu. Vlastnosti radu, ako napríklad jeho cestu a štatistiky, je možné aj jednoducho vykresliť.

```
// vytvorenie animovaného radu od bodu 0,0 po 500,0 s časom presunu 10
AnimQueue queue = new AnimQueue(animator, 0, 0, 500, 0, 10);
// vloženie animačného objektu do radu
queue.insert(item);
// vybratie z radu
```

```
queue.removeFirst();
// vybratie objektu z radu
queue.remove(item);
// zobrazenie vlastností radu
queue.setVisible(true);
```

Rozšírený režim

Náš animátor umožňuje používateľom rozšírenie a úpravu jeho častí dvoma spôsobmi. Prvým je implementácia vlastného animátora cez rozhranie *IAnimator*. Táto možnosť poskytuje plnú kontrolu nad animáciou, vykresľovaním a synchronizáciou, no vyžaduje kompletnú implementáciu všetkých častí jadra.

Druhou, jednoduchšou cestou je úprava len vybraných častí existujúceho animátora. Napríklad, ak by používateľ chcel upraviť alebo doplniť len vykresľovanie niektorej časti animačného jadra stačí mu vytvoriť vlastné jadro dediacu triedu *Animator* a prekryť jednu z metód *drawTransformed* (vykresľovanie po transformácií posunu a priblíženia plátna), *drawAnims* (vykreslenie animačných aktivít), *drawStatic* (statické vykresľovanie bez transformácie, napríklad na používateľské rozhranie) alebo *drawBackground*, *drawBackgroundImage* (kreslenie pozadia). Rovnako môže používateľ prekryť a upraviť aj časti mimo animačného jadra, ako napríklad vytvoriť nový typ animačného objektu s vlastným vykresľovaním alebo inou upravenou časťou a animačné jadro sa postará o jeho vykreslenie a synchronizáciu. Používateľ si tak môže prispôsobiť len konkrétnu funkcionalitu bez nutnosti zásahov do celého systému. Možné je aj vytváranie nových animačných objektov s vlastným správaním, pričom jadro sa automaticky postará o ich vykresľovanie a synchronizáciu.

Ovládanie plátna

Zobrazovaný stav simulácie sa nemusí zmestiť do zobrazeného priestoru pre plátno, preto plátno umožňuje posúvanie obrazu a jeho zmenšenie, zväčšenie.

Posúvanie obrazu: držanie ľavého tlačidla myši + posúvanie myšou

Zmenšenie veľkosti obrazu: držanie pravého tlačidla myši + posun myšou doľava Zväčšenie veľkosti obrazu: držanie pravého tlačidla myši + posun myšou doprava

Otvorenie menu objektu: stlačenie pravého tlačidla myši nad objektom

Súradnicový systém pre animačné objekty v plátne

Súradnicový systém je rovnaký ako pre ostatné Java komponenty. Ak ste neposunuli obrazom, alebo nezmenili jeho veľkosť, bod X:0, Y:0 bude v ľavom hornom rohu. Os Y naberá kladné hodnoty smerom dole. Os X naberá kladné hodnoty smerom doprava. Rovnako to platí pre C# verziu.

Záznam animácie

Animátor podporuje aj nahrávanie záznamu animácie. Tento záznam sa ukladá do binárneho súboru. Ukladajú sa všetky objekty a zmeny objektov, vrátane obrázkov a pozadia. Uložený záznam je možné kedykoľvek otvoriť aj v inom projekte, no nie je kompatibilný medzi Java a C# verziou. Spustenie a vypnutie nahrávania, vrátane otvorenia záznamu, je možné cez vstavané bočné menu. Otvorenie záznamu je možné aj potiahnutím a pustením súboru nad oknom animátora.

```
// spustenie nahrávania do súboru file.bin
animator.record(new File("file.bin"));

// ukončenie nahrávania
animator.stopRecording();

// prehratie záznamu
animator.replay(new File("file.bin"));
```

Mapový animátor

Vytvorenie a vloženie do GUI

Mapový animátor sa vytvára a vkladá do grafického rozhrania rovnakým spôsobom ako klasický animátor. Jediným rozdielom je potreba vytvoriť objekt *GISAnimator*. Po vložení do grafického rozhrania by sa mala zobraziť mapa vycentrovaná na Slovensko.

```
// vytvorenie mapového animátora
sim.setAnimator(new GISAnimator(sim));
```

Menu mapového animátora

Bočné menu animátora je rozšírené o možnosti nastavenia priblíženia na súradnice, menu vrstiev, pridávania bodov a exportu do obrázka.



Nastavenie mapového animátora

V prípade mapového animátora sú nastavenia, ako napríklad zmena pozadia, nahradené možnosťami na nastavenie mapy. Nastavenia mapy a vrstiev je možné meniť aj cez grafické rozhranie.

Používateľ má k dispozícii nasledujúce možnosti prispôsobenia:

```
// vycentrovanie mapy na určité súradnice a priblíženie
gisAnimator.centerMapToCoords(49.0681736, 19.3166392, 18);
// farba pozadia mapy
gisAnimator.setBackground(Color.white);
// priečinok, ktorý animátor bude požívať na ukladanie konfigurácie a
vrstiev
// defaultne priečinok projektu
gisAnimator.setFilesLocation(new File("/path/inputFiles/"));
// povoliť ukladanie mapových dlaždíc do súboru a RAM
// defaultne true, true
gisAnimator.setCacheTiles(true,true);
// nastavenie limitovania veľkosti ukladaných dát
// defaultne true, 1000MB
gisAnimator.setCacheSizeLimit(true,1000);
```

Práca s vrstvami mapy

```
// vytvorenie novej vrstvy OpenStreetMap
Layer layer = new Layer("OpenStreetMap",
"https://tile.openstreetmap.org/${z}/${x}/${y}.png", "WMTS", "c
Prispievatelia OpenStreetMap", "link");
// pridanie novej vrstvy
gisAnimator.addLayer(layer);
// prepínanie medzi uloženými vrstvami
gisAnimator.setLayer(1);
gisAnimator.setLayer(layer);

// pridanie a prepínanie dodatočných vrstiev
gisAnimator.addSecondaryLayer(layer);
gisAnimator.setSecondarLayer(layer);
```

Animačné objekty

Práca s animačnými objektami (vytvorenie/registrácia) pre mapový animátor je rovnaká ako pre základný animátor. Mapový animátor podporuje aj základné animačné objekty, ktoré sa zobrazujú staticky na obrazovke. Pre použitie animačných objektov s geografickými súradnicami je potrebné využiť objekty *GISAnimImageItem*, *GISAnimShapeItem* a *GISAnimTextItem*. Tieto objekty automaticky pracujú s geografickými súradnicami a navyše

podporujú funkcie ako nastavenie veľkosti v metroch. Tieto animačné objekty navyše podporujú aj použitie *osmId* namiesto súradníc.

Možnosti navyše:

```
// určujte či veľkosť animačného objektu je v px alebo metroch
// true - px, false - metre
// defaultne false
item.setStaticSize(false);

// určujte či aj Label sa škáluje spolu s animačným objektom
// defaultne false
item.setScaleLabel(false);
```

Súradnice mapového animátora

Mapový animátor pracuje so súradnicami WGS84 vo formáte desatinných stupňov, napríklad: *49.1903858, 19.0090219*. Na ich použitie je možné využiť triedu *Coordinates* alebo *Point2D*, kde X predstavuje zemepisnú šírku (*latitude*) a Y zemepisnú dĺžku (*longitude*).

Práca s dátami OpenStreetMap

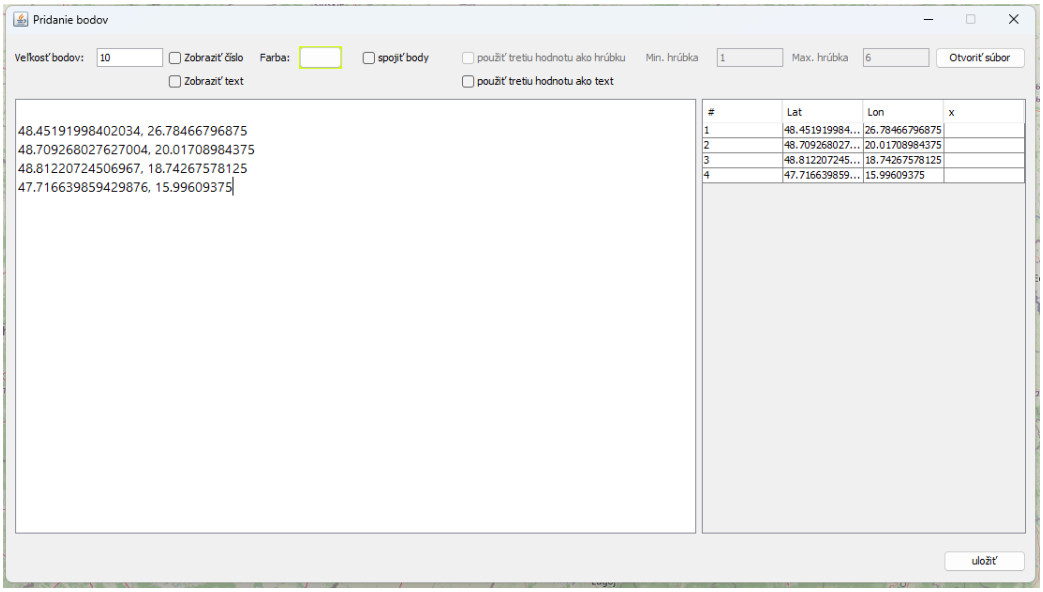
Mapový animátor vie pracovať aj s dátami z projektu OpenStreetMap. Aby bolo možné s týmito dátami pracovať, je potrebné ich vložiť ako *.osm* súbor. Tento súbor je možné exportovať pomocou rôznych online služieb. Po jeho spracovaní je možné namiesto geografických súradníc používať body identifikované pomocou *osmId*. Súbor je možné využiť aj na hľadanie najkratších ciest z bodu A do bodu B.

```
// spracovanie dát XML OSM
// je potrebné spracovať pred použitím pri animácií
OSMTools.parseOSMFile("../Restauracia Source/export.osm");

// príklad použitia na nájdenie najkratšej cesty
List<Coordinates> cesta = OSMTools.findShortestPath(suradnice,
suradnice2);
animObject.startAnim(mySim().currentTime(), trvaniePresunu, cesta);
```

Pridávanie bodov

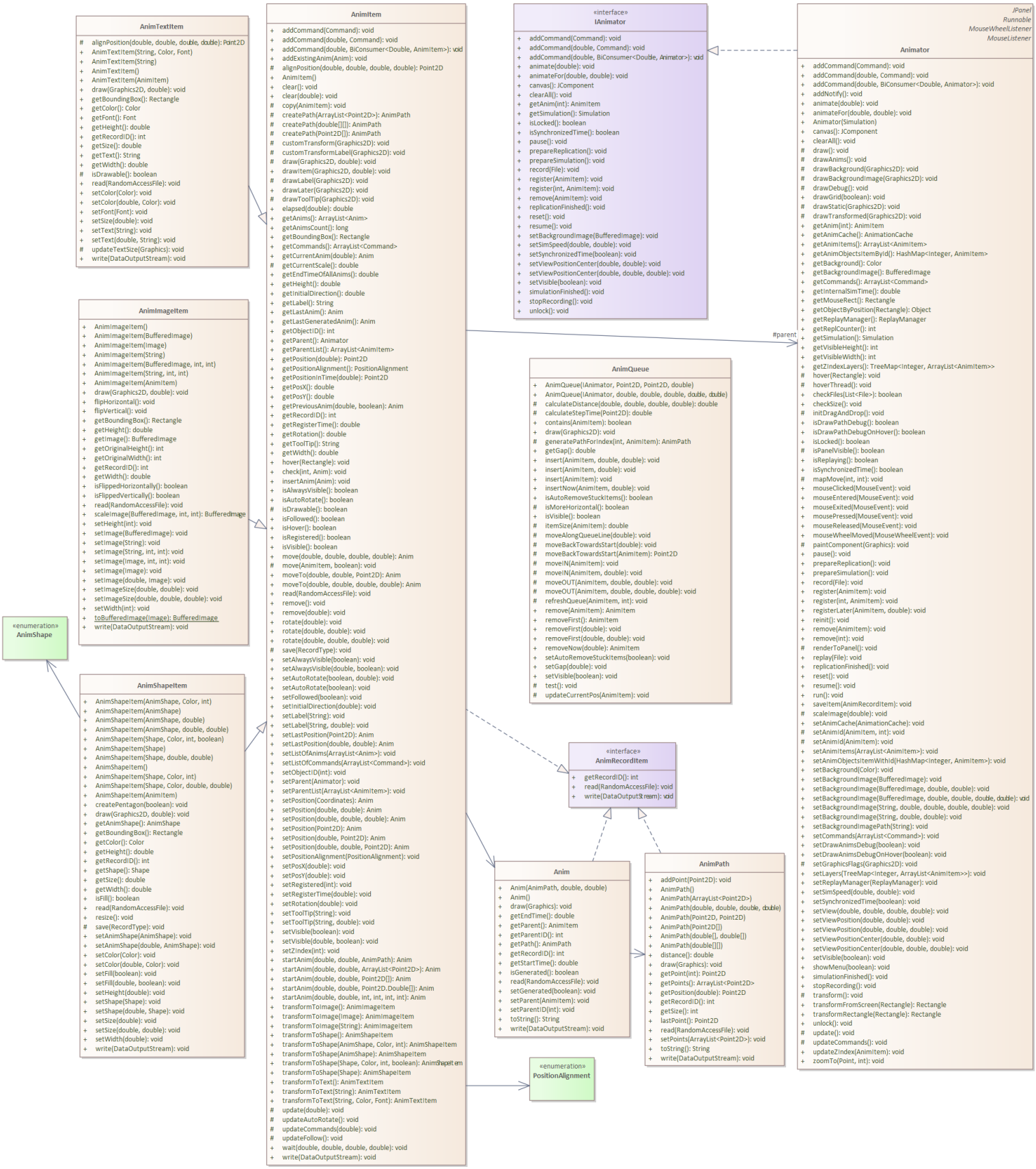
Mapový animátor obsahuje aj nástroj na grafické pridávanie a správu bodov. Tento nástroj je možné otvoriť kliknutím na príslušnú ikonu v hlavnom grafickom menu. Na pridanie bodu stačí zadať geografické súradnice do textového poľa. Súradnice sú automaticky rozpoznané a po uložení sa zobrazia na mape. Body je možné pridať aj pravým kliknutím na mapu a zvolením možnosti „Pridať bod“. Body je možné jednoducho vložiť aj prostredníctvom programátorského rozhrania alebo ich načítať zo súboru. Zobrazené body je následne možné prispôsobiť. Používateľ môže napríklad ľubovoľne nastaviť ich veľkosť, farbu, pridať text alebo zobraziť poradové čísla bodov. Body je taktiež možné medzi sebou pospájať. V tomto prípade sa medzi nimi vykreslí čiara s nastaviteľnou hrúbkou.

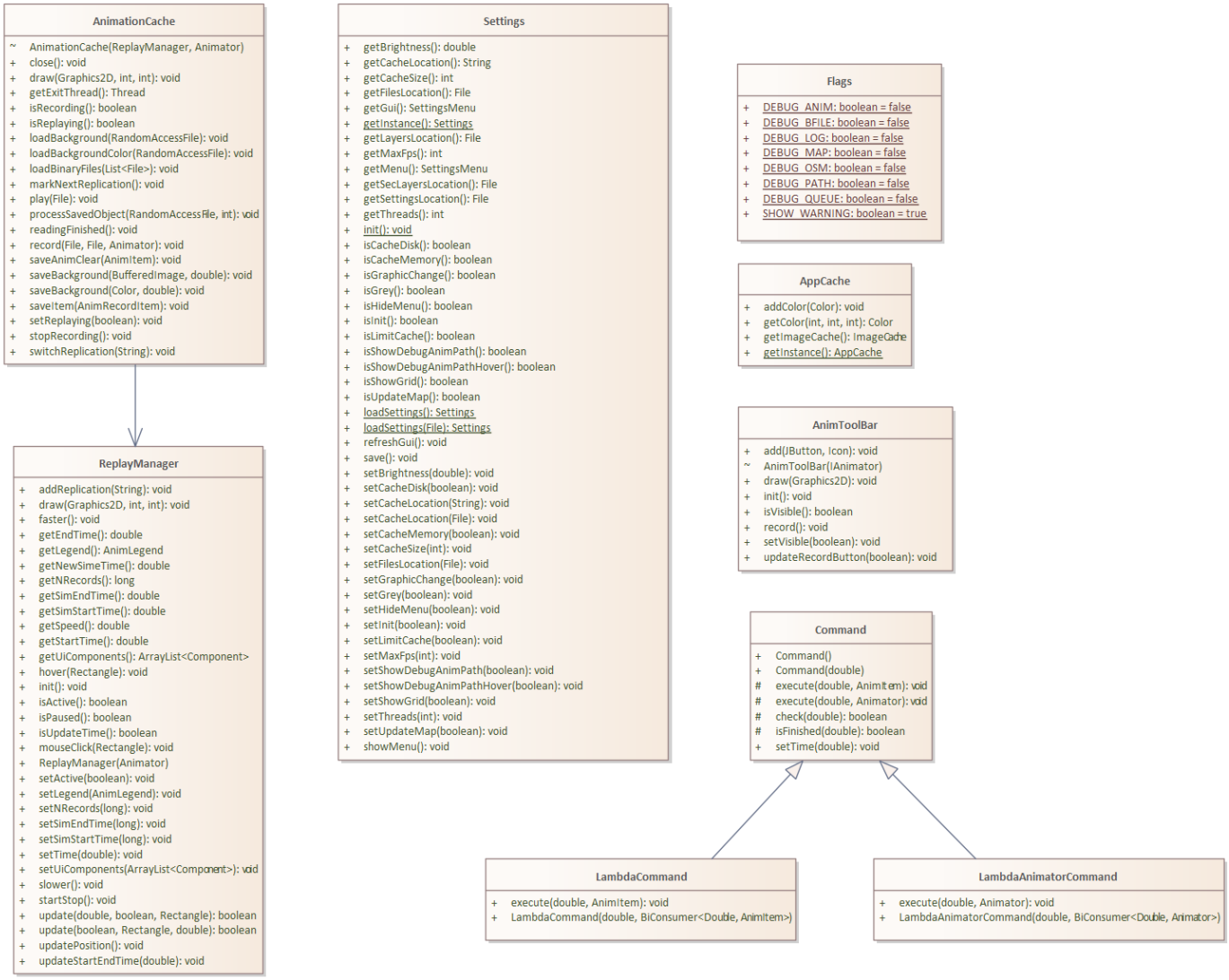


Odporúčania

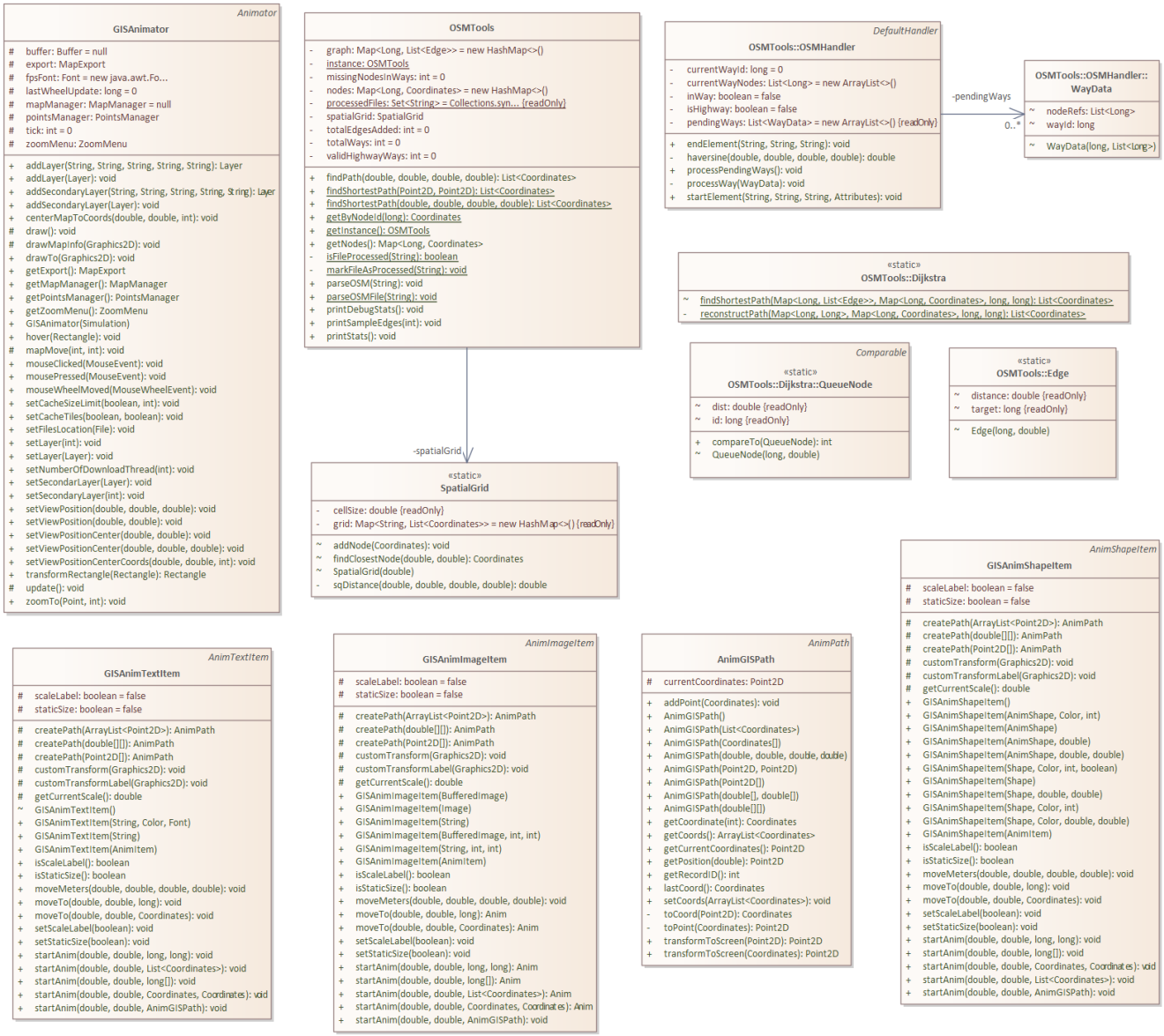
- Odporúča sa dôsledne mazať nepoužité animačné objekty pomocou *anim.remove(),* keď sa prestanú používať, alebo na konci replikácie pomocou *animator.clear().*
- Ak má objekt nastavenú pozíciu/cestu v danom simulačnom čase, ale nezobrazuje sa, odporúča sa skontrolovať, či objekt je registrovaný - *anim.isRegistered().*
- Animačný objekt by mal mať v určitom čase iba jednu animáciu. Viaceré prekrývajúce sa animácie môžu spôsobiť rôzne problémy, napríklad skákanie z miesta na miesto alebo návrat na starú pozíciu.
- Ak vznikajú v animovanom rade medzery, pravdepodobne sa animácie nevyberajú v poradí, v akom boli vložené. V takom prípade sa odporúča použiť *remove(anim)* namiesto *removeFirst().*
- Označenie „Zobraziť štatistiky animátora“ v nastaveniach alebo nastavenie statickej premennej *Flags.DEBUG_ANIM = true* zobrazí informácie ako FPS, počet aktuálne registrovaných animácií a pod.
- Ak nefunguje *setViewPositionCenter* alebo jeho alternatívy, pravdepodobne animátor v čase volania ešte nemá nastavenú veľkosť. Odporúča sa tieto metódy používať až po vložení animátora do grafického rozhrania a po nastavení jeho veľkosti.
- Nesprávna veľkosť animátora môže byť spôsobená nesprávne nastaveným grafickým rozhraním, do ktorého sa animátor vkladá. Odporúča sa preto skontrolovať nastavenia rodičovského prvku (*layout...*).

Diagram hlavných častí animátora

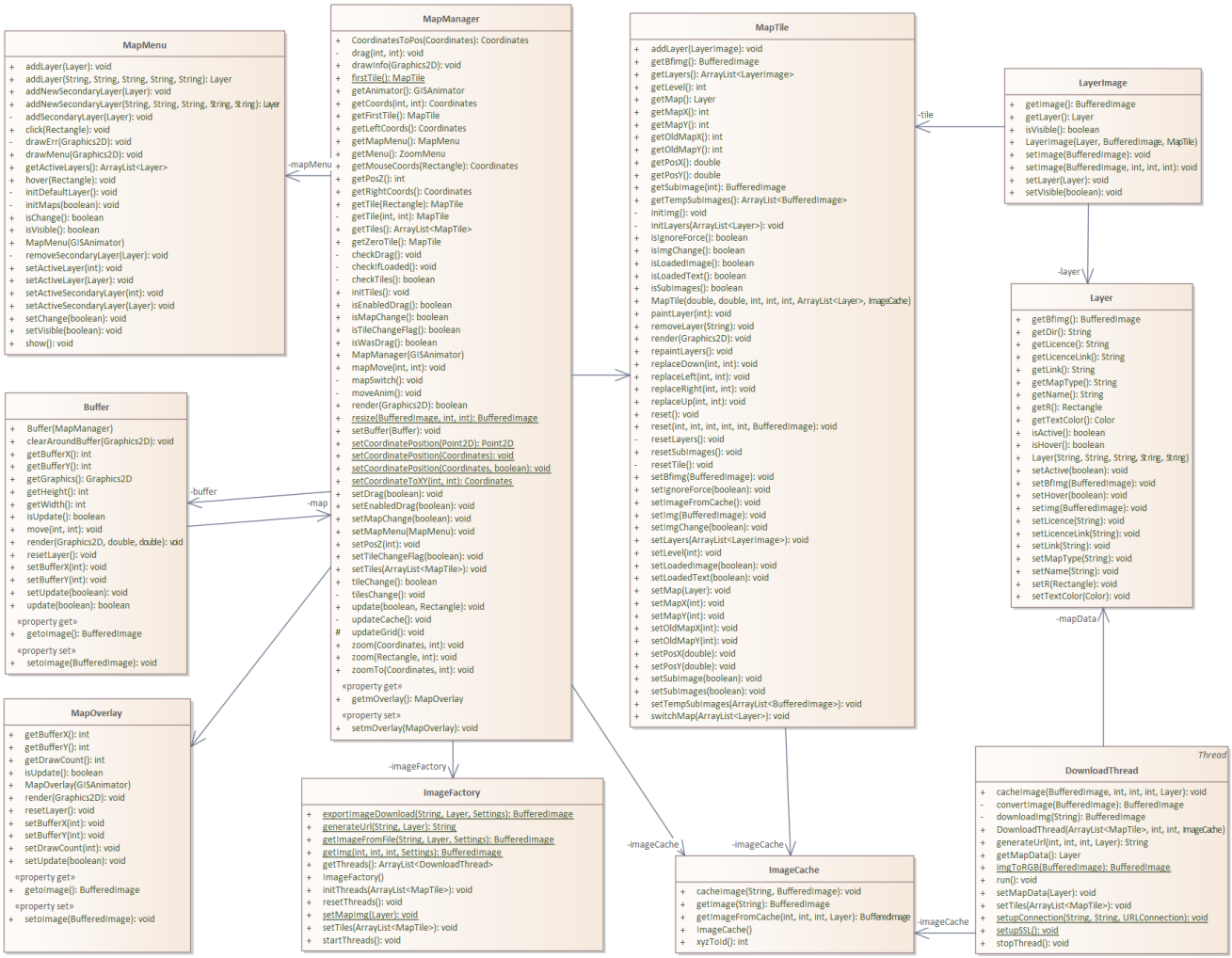

AnimTextItem
alignPosition(double, double, double, double): Point2D
+ AnimTextItem(String, Color, Font)
+ AnimTextItem(String)
+ AnimTextItem()
+ AnimTextItem(AnimItem)
+ draw(Graphics2D, double): void
+ getBoundingBox(): Rectangle
+ getColor(): Color
+ getFont(): Font
+ getHeight(): double
+ getRecordID(): int
+ getSize(): double
+ getText(): String
+ getWidth(): double
isDrawable(): boolean
+ read(RandomAccessFile): void
+ setColor(Color): void
+ setColor(double, Color): void
+ setFont(Font): void
+ setSize(double): void
+ setText(String): void
+ setText(double, String): void
updateTextSize(Graphics): void
+ write(DataOutputStream): void
AnimItem
+ addCommand(Command): void
+ addCommand(double, Command): void
+ addCommand(double, BiConsumer<Double, AnimItem>): void
+ addExistingAnim(Anim): void
alignPosition(double, double, double, double): Point2D
+ AnimItem()
+ clear(): void
+ clear(double): void
copy(AnimItem): void
createPath(ArrayList<Point2D>): AnimPath
createPath(double[][]): AnimPath
createPath(Point2D[]): AnimPath
customTransform(Graphics2D): void
customTransformLabel(Graphics2D): void
draw(Graphics2D, double): void
+ drawItem(Graphics2D, double): void
drawLabel(Graphics2D): void
+ drawLater(Graphics2D): void
drawToolTip(Graphics2D): void
+ elapsed(double): double
+ getAnims(): ArrayList<Anim>
+ getAnimsCount(): long
+ getBoundingBox(): Rectangle
+ getCommands(): ArrayList<Command>
+ getCurrentAnim(double): Anim
getCurrentScale(): double
+ getEndTimeOfAllAnims(): double
+ getHeight(): double
+ getInitialDirection(): double
+ getLabel(): String
+ getLastAnim(): Anim
+ getLastGeneratedAnim(): Anim
+ getObjectID(): int
+ getParent(): Animator
+ getParentList(): ArrayList<AnimItem>
+ getPosition(double): Point2D
+ getPositionAlignment(): PositionAlignment
+ getPositionInTime(double): Point2D
+ getPosX(): double
+ getPosY(): double
+ getPreviousAnim(double, boolean): Anim
+ getRecordID(): int
+ getRegisterTime(): double
+ getRotation(): double
+ getToolTip(): String
+ getWidth(): double
+ hover(Rectangle): void
+ check(int, Anim): void
+ insertAnim(Anim): void
+ isAlwaysVisible(): boolean
+ isAutoRotate(): boolean
isDrawable(): boolean
+ isFollowed(): boolean
+ isHover(): boolean
+ isRegistered(): boolean
+ isVisible(): boolean
+ move(double, double, double, double): Anim
move(AnimItem, boolean): void
+ moveTo(double, double, Point2D): Anim
+ moveTo(double, double, double, double): Anim
+ read(RandomAccessFile): void
+ remove(): void
+ remove(double): void
+ rotate(double): void
+ rotate(double, double): void
+ rotate(double, double, double): void
save(RecordType): void
+ setAlwaysVisible(boolean): void
+ setAlwaysVisible(double, boolean): void
setAutoRotate(boolean, double): void
+ setAutoRotate(boolean): void
+ setFollowed(boolean): void
+ setInitialDirection(double): void
+ setLabel(String): void
+ setLabel(String, double): void
+ setLastPosition(Point2D): Anim
+ setLastPosition(double, double): Anim
+ setListOfAnims(ArrayList<Anim>): void
+ setListOfCommands(ArrayList<Command>): void
+ setObjectID(int): void
+ setParent(Animator): void
+ setParentList(ArrayList<AnimItem>): void
+ setPosition(Coordinates): Anim
+ setPosition(double, double): Anim
+ setPosition(double, double, double): Anim
+ setPosition(Point2D): Anim
+ setPosition(double, Point2D): Anim
+ setPosition(double, double, Point2D): Anim
+ setPositionAlignment(PositionAlignment): void
+ setPosX(double): void
+ setPosY(double): void
+ setRegistered(int): void
+ setRegisterTime(double): void
+ setRotation(double): void
+ setToolTip(String): void
+ setToolTip(String, double): void
+ setVisible(boolean): void
+ setVisible(double, boolean): void
+ setZIndex(int): void
+ startAnim(double, double, AnimPath): Anim
+ startAnim(double, double, ArrayList<Point2D>): Anim
+ startAnim(double, double, Point2D[]): Anim
+ startAnim(double, double, Point2D.Double[]): Anim
+ startAnim(double, double, int, int, int, int): Anim
+ transformToImage(): AnimImageItem
+ transformToImage(Image): AnimImageItem
+ transformToImage(String): AnimImageItem
+ transformToShape(): AnimShapeItem
+ transformToShape(AnimShape, Color, int): AnimShapeItem
+ transformToShape(AnimShape): AnimShapeItem
+ transformToShape(Shape, Color, int, boolean): AnimShapeItem
+ transformToShape(Shape): AnimShapeItem
+ transformToText(): AnimTextItem
+ transformToText(String): AnimTextItem
+ transformToText(String, Color, Font): AnimTextItem
update(double): void
updateAutoRotate(): void
updateCommands(double): void
updateFollow(): void
+ wait(double, double, double, double): void
+ write(DataOutputStream): void
«interface»
IAnimator
+ addCommand(Command): void
+ addCommand(double, Command): void
+ addCommand(double, BiConsumer<Double, Animator>): void
+ animate(double): void
+ animateFor(double, double): void
+ canvas(): JComponent
+ clearAll(): void
+ getAnim(int): AnimItem
+ getSimulation(): Simulation
+ isLocked(): boolean
+ isSynchronizedTime(): boolean
+ pause(): void
+ prepareReplication(): void
+ prepareSimulation(): void
+ record(File): void
+ register(AnimItem): void
+ register(int, AnimItem): void
+ remove(AnimItem): void
+ replicationFinished(): void
+ reset(): void
+ resume(): void
+ setBackgroundImage(BufferedImage): void
+ setSimSpeed(double, double): void
+ setSynchronizedTime(boolean): void
+ setViewPositionCenter(double, double): void
+ setViewPositionCenter(double, double, double): void
+ setVisible(boolean): void
+ simulationFinished(): void
+ stopRecording(): void
+ unlock(): void
JPanel
Runnable
MouseWheelListener
MouseListener
Animator
+ addCommand(Command): void
+ addCommand(double, Command): void
+ addCommand(double, BiConsumer<Double, Animator>): void
+ addNotify(): void
+ animate(double): void
+ animateFor(double, double): void
+ Animator(Simulation)
+ canvas(): JComponent
+ clearAll(): void
draw(): void
drawAnims(): void
drawBackground(Graphics2D): void
drawBackgroundImage(Graphics2D): void
drawDebug(): void
+ drawGrid(boolean): void
drawStatic(Graphics2D): void
drawTransformed(Graphics2D): void
+ getAnim(int): AnimItem
+ getAnimCache(): AnimationCache
+ getAnimItems(): ArrayList<AnimItem>
+ getAnimObjectsItemById(): HashMap<Integer, AnimItem>
+ getBackground(): Color
+ getBackgroundImage(): BufferedImage
+ getCommands(): ArrayList<Command>
+ getInternalSimTime(): double
+ getMouseRect(): Rectangle
+ getObjectByPosition(Rectangle): Object
+ getReplayManager(): ReplayManager
+ getReplCounter(): int
+ getSimulation(): Simulation
+ getVisibleHeight(): int
+ getVisibleWidth(): int
+ getZIndexLayers(): TreeMap<Integer, ArrayList<AnimItem>>
hover(Rectangle): void
hoverThread(): void
+ checkFiles(List<File>): boolean
+ checkSize(): void
initDragAndDrop(): void
+ isDrawPathDebug(): boolean
+ isDrawPathDebugOnHover(): boolean
+ isLocked(): boolean
isPanelVisible(): boolean
+ isReplaying(): boolean
+ isSynchronizedTime(): boolean
mapMove(int, int): void
+ mouseClicked(MouseEvent): void
+ mouseEntered(MouseEvent): void
+ mouseExited(MouseEvent): void
+ mousePressed(MouseEvent): void
+ mouseReleased(MouseEvent): void
+ mouseWheelMoved(MouseWheelEvent): void
paintComponent(Graphics): void
+ pause(): void
+ prepareReplication(): void
+ prepareSimulation(): void
+ record(File): void
+ register(AnimItem): void
+ register(int, AnimItem): void
+ registerLater(AnimItem, double): void
+ reinit(): void
+ remove(AnimItem): void
+ remove(int): void
renderToPanel(): void
+ replay(File): void
+ replicationFinished(): void
+ reset(): void
+ resume(): void
+ run(): void
+ saveItem(AnimRecordItem): void
scaleImage(double): void
+ setAnimCache(AnimationCache): void
setAnimId(AnimItem, int): void
setAnimId(AnimItem): void
+ setAnimItems(ArrayList<AnimItem>): void
+ setAnimObjectsItemWithId(HashMap<Integer, AnimItem>): void
+ setBackground(Color): void
+ setBackgroundImage(BufferedImage): void
+ setBackgroundImage(BufferedImage, double, double): void
+ setBackgroundImage(BufferedImage, double, double, double, double): void
+ setBackgroundImage(String, double, double, double): void
+ setBackgroundImage(String, double, double): void
+ setBackgroundImagePath(String): void
+ setCommands(ArrayList<Command>): void
+ setDrawAnimsDebug(boolean): void
+ setDrawAnimsDebugOnHover(boolean): void
setGraphicsFlags(Graphics2D): void
+ setLayers(TreeMap<Integer, ArrayList<AnimItem>>): void
+ setReplayManager(ReplayManager): void
+ setSimSpeed(double, double): void
+ setSynchronizedTime(boolean): void
+ setView(double, double, double, double): void
+ setViewPosition(double, double): void
+ setViewPosition(double, double, double): void
+ setViewPositionCenter(double, double): void
+ setViewPositionCenter(double, double, double): void
+ setVisible(boolean): void
+ showMenu(boolean): void
+ simulationFinished(): void
+ stopRecording(): void
transform(): void
+ transformFromScreen(Rectangle): Rectangle
+ transformRectangle(Rectangle): Rectangle
+ unlock(): void
update(): void
updateCommands(): void
+ updateZIndex(AnimItem): void
+ zoomTo(Point, int): void
#parent
AnimQueue
+ AnimQueue(IAnimator, Point2D, Point2D, double)
+ AnimQueue(IAnimator, double, double, double, double, double)
calculateDistance(double, double, double, double): double
calculateStepTime(Point2D): double
+ contains(AnimItem): boolean
+ draw(Graphics2D): void
generatePathForIndex(int, AnimItem): AnimPath
+ getGap(): double
+ insert(AnimItem, double, double): void
+ insert(AnimItem, double): void
+ insert(AnimItem): void
+ insertNow(AnimItem, double): void
+ isAutoRemoveStuckItems(): boolean
isMoreHorizontal(): boolean
+ isVisible(): boolean
itemSize(AnimItem): double
moveAlongQueueLine(double): void
moveBackTowardsStart(double): void
moveBackTowardsStart(AnimItem): Point2D
moveIN(AnimItem): void
moveIN(AnimItem, double): void
moveOUT(AnimItem, double, double): void
moveOUT(AnimItem, double, double, double): void
refreshQueue(AnimItem, int): void
+ remove(AnimItem): AnimItem
+ removeFirst(): AnimItem
+ removeFirst(double): void
+ removeFirst(double, double): void
+ removeNow(double): AnimItem
+ setAutoRemoveStuckItems(boolean): void
+ setGap(double): void
+ setVisible(boolean): void
test(): void
updateCurrentPos(AnimItem): void
«interface»
AnimRecordItem
+ getRecordID(): int
+ read(RandomAccessFile): void
+ write(DataOutputStream): void
Anim
+ Anim(AnimPath, double, double)
+ Anim()
+ draw(Graphics): void
+ getEndTime(): double
+ getParent(): AnimItem
+ getParentID(): int
+ getPath(): AnimPath
+ getRecordID(): int
+ getStartTime(): double
+ isGenerated(): boolean
+ read(RandomAccessFile): void
+ setGenerated(boolean): void
+ setParent(AnimItem): void
+ setParentID(int): void
+ toString(): String
+ write(DataOutputStream): void
AnimPath
+ addPoint(Point2D): void
+ AnimPath()
+ AnimPath(ArrayList<Point2D>)
+ AnimPath(double, double, double, double)
+ AnimPath(Point2D, Point2D)
+ AnimPath(Point2D[])
+ AnimPath(double[], double[])
+ AnimPath(double[][])
+ distance(): double
+ draw(Graphics): void
+ getPoint(int): Point2D
+ getPoints(): ArrayList<Point2D>
+ getPosition(double): Point2D
+ getRecordID(): int
+ getSize(): int
+ lastPoint(): Point2D
+ read(RandomAccessFile): void
+ setPoints(ArrayList<Point2D>): void
+ toString(): String
+ write(DataOutputStream): void
«enumeration»
AnimShape
AnimShapeItem
+ AnimShapeItem(AnimShape, Color, int)
+ AnimShapeItem(AnimShape)
+ AnimShapeItem(AnimShape, double)
+ AnimShapeItem(AnimShape, double, double)
+ AnimShapeItem(Shape, Color, int, boolean)
+ AnimShapeItem(Shape)
+ AnimShapeItem(Shape, double, double)
+ AnimShapeItem()
+ AnimShapeItem(Shape, Color, int)
+ AnimShapeItem(Shape, Color, double, double)
+ AnimShapeItem(AnimItem)
+ createPentagon(boolean): void
+ draw(Graphics2D, double): void
+ getAnimShape(): AnimShape
+ getBoundingBox(): Rectangle
+ getColor(): Color
+ getHeight(): double
+ getRecordID(): int
+ getShape(): Shape
+ getSize(): double
+ getWidth(): double
+ isFill(): boolean
+ read(RandomAccessFile): void
+ resize(): void
save(RecordType): void
+ setAnimShape(AnimShape): void
+ setAnimShape(double, AnimShape): void
+ setColor(Color): void
+ setColor(double, Color): void
+ setFill(boolean): void
+ setFill(double, boolean): void
+ setHeight(double): void
+ setShape(Shape): void
+ setShape(double, Shape): void
+ setSize(double): void
+ setSize(double, double): void
+ setWidth(double): void
+ write(DataOutputStream): void
AnimImageItem
+ AnimImageItem()
+ AnimImageItem(BufferedImage)
+ AnimImageItem(Image)
+ AnimImageItem(String)
+ AnimImageItem(BufferedImage, int, int)
+ AnimImageItem(String, int, int)
+ AnimImageItem(AnimItem)
+ draw(Graphics2D, double): void
+ flipHorizontal(): void
+ flipVertical(): void
+ getBoundingBox(): Rectangle
+ getHeight(): double
+ getImage(): BufferedImage
+ getOriginalHeight(): int
+ getOriginalWidth(): int
+ getRecordID(): int
+ getWidth(): double
+ isFlippedHorizontally(): boolean
+ isFlippedVertically(): boolean
+ read(RandomAccessFile): void
+ scaleImage(BufferedImage, int, int): BufferedImage
+ setHeight(int): void
+ setImage(BufferedImage): void
+ setImage(String): void
+ setImage(String, int, int): void
+ setImage(Image, int, int): void
+ setImage(Image): void
+ setImage(double, Image): void
+ setImageSize(double, double): void
+ setImageSize(double, double, double): void
+ setWidth(int): void
+ toBufferedImage(Image): BufferedImage
+ write(DataOutputStream): void
«enumeration»
PositionAlignment



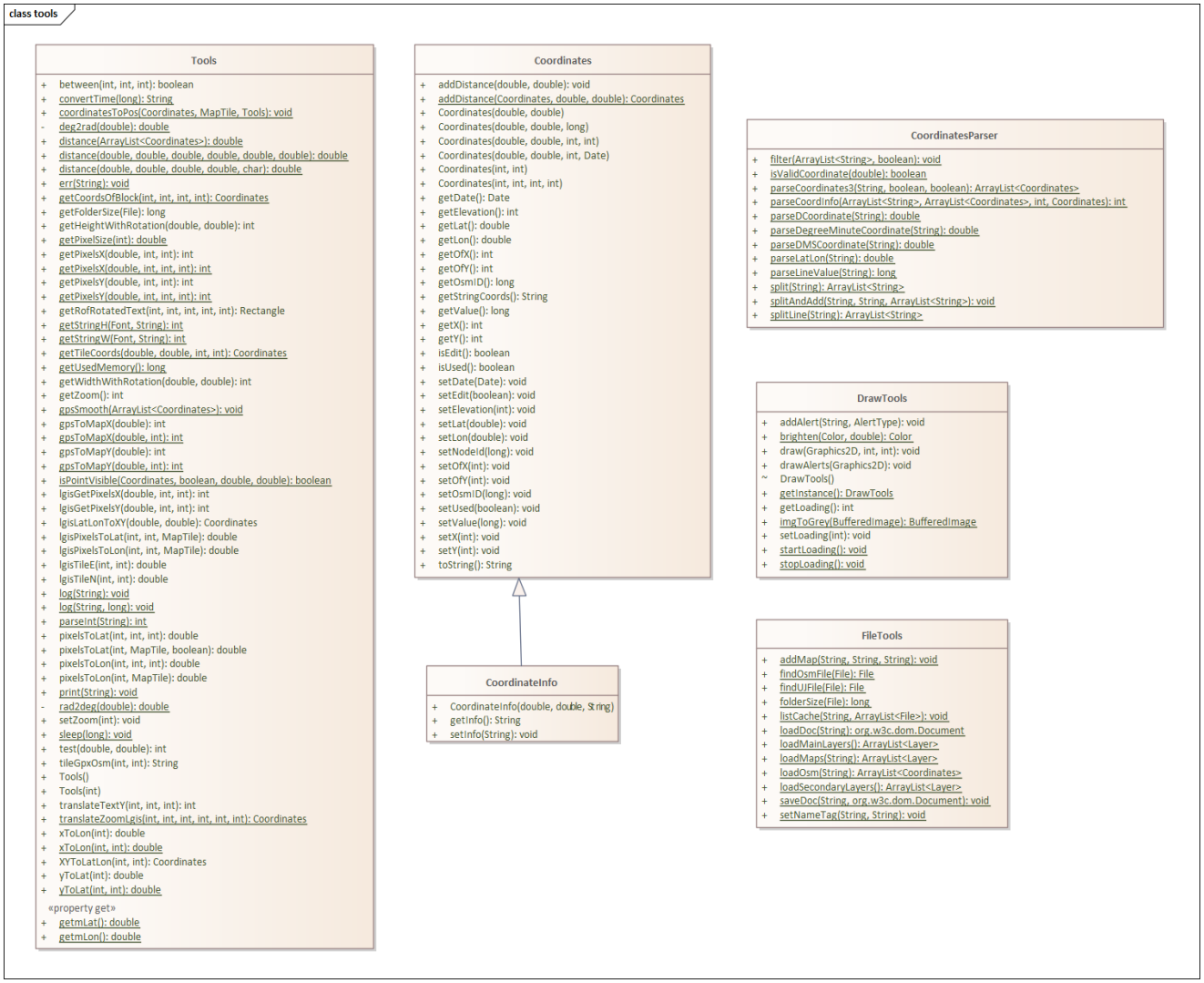
Mapový animátor

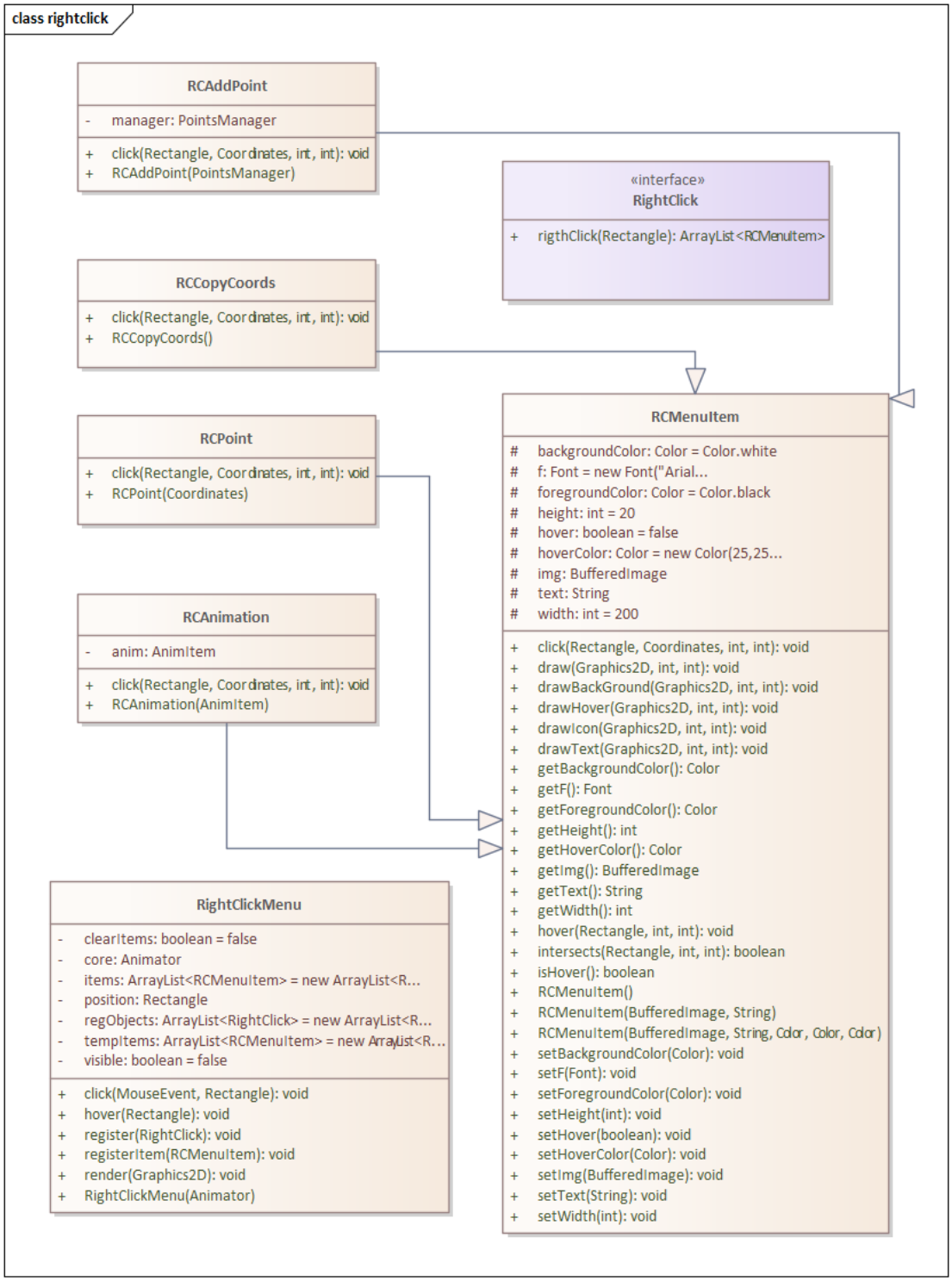


Mapové zobrazenie


MapMenu
+ addLayer(Layer): void
+ addLayer(String, String, String, String, String): Layer
+ addNewSecondaryLayer(Layer): void
+ addNewSecondaryLayer(String, String, String, String, String): Layer
- addSecondaryLayer(Layer): void
+ click(Rectangle): void
- drawErr(Graphics2D): void
+ drawMenu(Graphics2D): void
+ getActiveLayers(): ArrayList<Layer>
+ hover(Rectangle): void
- initDefaultLayer(): void
- initMaps(boolean): void
+ isChange(): boolean
+ isVisible(): boolean
+ MapMenu(GISAnimator)
- removeSecondaryLayer(Layer): void
+ setActiveLayer(int): void
+ setActiveLayer(Layer): void
+ setActiveSecondaryLayer(int): void
+ setActiveSecondaryLayer(Layer): void
+ setChange(boolean): void
+ setVisible(boolean): void
+ show(): void
-mapMenu
MapManager
+ CoordinatesToPos(Coordinates): Coordinates
- drag(int, int): void
+ drawInfo(Graphics2D): void
+ firstTile(): MapTile
+ getAnimator(): GISAnimator
+ getCoords(int, int): Coordinates
+ getFirstTile(): MapTile
+ getLeftCoords(): Coordinates
+ getMapMenu(): MapMenu
+ getMenu(): ZoomMenu
+ getMouseCoords(Rectangle): Coordinates
+ getPosZ(): int
+ getRightCoords(): Coordinates
+ getTile(Rectangle): MapTile
- getTile(int, int): MapTile
+ getTiles(): ArrayList<MapTile>
+ getZeroTile(): MapTile
- checkDrag(): void
- checkIfLoaded(): void
- checkTiles(): boolean
+ initTiles(): void
+ isEnabledDrag(): boolean
+ isMapChange(): boolean
+ isTileChangeFlag(): boolean
+ isWasDrag(): boolean
+ MapManager(GISAnimator)
+ mapMove(int, int): void
- mapSwitch(): void
- moveAnim(): void
+ render(Graphics2D): boolean
+ resize(BufferedImage, int, int): BufferedImage
+ setBuffer(Buffer): void
+ setCoordinatePosition(Point2D): Point2D
+ setCoordinatePosition(Coordinates): void
+ setCoordinatePosition(Coordinates, boolean): void
+ setCoordinateToXY(int, int): Coordinates
+ setDrag(boolean): void
+ setEnabledDrag(boolean): void
+ setMapChange(boolean): void
+ setMapMenu(MapMenu): void
+ setPosZ(int): void
+ setTileChangeFlag(boolean): void
+ setTiles(ArrayList<MapTile>): void
+ tileChange(): boolean
- tilesChange(): void
+ update(boolean, Rectangle): void
- updateCache(): void
updateGrid(): void
+ zoom(Coordinates, int): void
+ zoom(Rectangle, int): void
+ zoomTo(Coordinates, int): void
«property get»
+ getmOverlay(): MapOverlay
«property set»
+ setmOverlay(MapOverlay): void
MapTile
+ addLayer(LayerImage): void
+ getBfimg(): BufferedImage
+ getLayers(): ArrayList<LayerImage>
+ getLevel(): int
+ getMap(): Layer
+ getMapX(): int
+ getMapY(): int
+ getOldMapX(): int
+ getOldMapY(): int
+ getPosX(): double
+ getPosY(): double
+ getSubImage(int): BufferedImage
+ getTempSubImages(): ArrayList<BufferedImage>
- initImg(): void
- initLayers(ArrayList<Layer>): void
+ isIgnoreForce(): boolean
+ isImgChange(): boolean
+ isLoadedImage(): boolean
+ isLoadedText(): boolean
+ isSubImages(): boolean
+ MapTile(double, double, int, int, int, ArrayList<Layer>, ImageCache)
+ paintLayer(int): void
+ removeLayer(String): void
+ render(Graphics2D): void
+ repaintLayers(): void
+ replaceDown(int, int): void
+ replaceLeft(int, int): void
+ replaceRight(int, int): void
+ replaceUp(int, int): void
+ reset(): void
+ reset(int, int, int, int, int, BufferedImage): void
- resetLayers(): void
+ resetSubImages(): void
- resetTile(): void
+ setBfimg(BufferedImage): void
+ setIgnoreForce(boolean): void
+ setImageFromCache(): void
+ setImg(BufferedImage): void
+ setImgChange(boolean): void
+ setLayers(ArrayList<LayerImage>): void
+ setLevel(int): void
+ setLoadedImage(boolean): void
+ setLoadedText(boolean): void
+ setMap(Layer): void
+ setMapX(int): void
+ setMapY(int): void
+ setOldMapX(int): void
+ setOldMapY(int): void
+ setPosX(double): void
+ setPosY(double): void
+ setSubImage(boolean): void
+ setSubImages(boolean): void
+ setTempSubImages(ArrayList<BufferedImage>): void
+ switchMap(ArrayList<Layer>): void
-tile
LayerImage
+ getImage(): BufferedImage
+ getLayer(): Layer
+ isVisible(): boolean
+ LayerImage(Layer, BufferedImage, MapTile)
+ setImage(BufferedImage): void
+ setImage(BufferedImage, int, int, int): void
+ setLayer(Layer): void
+ setVisible(boolean): void
-layer
Layer
+ getBfimg(): BufferedImage
+ getDir(): String
+ getLicence(): String
+ getLicenceLink(): String
+ getLink(): String
+ getMapType(): String
+ getName(): String
+ getR(): Rectangle
+ getTextColor(): Color
+ isActive(): boolean
+ isHover(): boolean
+ Layer(String, String, String, String, String)
+ setActive(boolean): void
+ setBfimg(BufferedImage): void
+ setHover(boolean): void
+ setImg(BufferedImage): void
+ setLicence(String): void
+ setLicenceLink(String): void
+ setLink(String): void
+ setMapType(String): void
+ setName(String): void
+ setR(Rectangle): void
+ setTextColor(Color): void
-mapData
Buffer
+ Buffer(MapManager)
+ clearAroundBuffer(Graphics2D): void
+ getBufferX(): int
+ getBufferY(): int
+ getGraphics(): Graphics2D
+ getHeight(): int
+ getWidth(): int
+ isUpdate(): boolean
+ move(int, int): void
+ render(Graphics2D, double, double): void
+ resetLayer(): void
+ setBufferX(int): void
+ setBufferY(int): void
+ setUpdate(boolean): void
+ update(boolean): boolean
«property get»
+ getoImage(): BufferedImage
«property set»
+ setoImage(BufferedImage): void
-buffer
-map
MapOverlay
+ getBufferX(): int
+ getBufferY(): int
+ getDrawCount(): int
+ isUpdate(): boolean
+ MapOverlay(GISAnimator)
+ render(Graphics2D): void
+ resetLayer(): void
+ setBufferX(int): void
+ setBufferY(int): void
+ setDrawCount(int): void
+ setUpdate(boolean): void
«property get»
+ getoImage(): BufferedImage
«property set»
+ setoImage(BufferedImage): void
-imageFactory
ImageFactory
+ exportImageDownload(String, Layer, Settings): BufferedImage
+ generateUrl(String, Layer): String
+ getImageFromFile(String, Layer, Settings): BufferedImage
+ getImg(int, int, int, Settings): BufferedImage
+ getThreads(): ArrayList<DownloadThread>
+ ImageFactory()
+ initThreads(ArrayList<MapTile>): void
+ resetThreads(): void
+ setMapImg(Layer): void
+ setTiles(ArrayList<MapTile>): void
+ startThreads(): void
-imageCache
-imageCache
ImageCache
+ cacheImage(String, BufferedImage): void
+ getImage(String): BufferedImage
+ getImageFromCache(int, int, int, Layer): BufferedImage
+ ImageCache()
+ xyzToId(): int
-imageCache
Thread
DownloadThread
+ cacheImage(BufferedImage, int, int, int, Layer): void
- convertImage(BufferedImage): BufferedImage
- downloadImg(String): BufferedImage
+ DownloadThread(ArrayList<MapTile>, int, int, ImageCache)
+ generateUrl(int, int, int, Layer): String
+ getMapData(): Layer
+ imgToRGB(BufferedImage): BufferedImage
+ run(): void
+ setMapData(Layer): void
+ setTiles(ArrayList<MapTile>): void
+ setupConnection(String, String, URLConnection): void
+ setupSSL(): void
+ stopThread(): void

Nástroje


class tools
Tools
+ between(int, int, int): boolean
+ convertTime(long): String
+ coordinatesToPos(Coordinates, MapTile, Tools): void
- deg2rad(double): double
+ distance(ArrayList<Coordinates>): double
+ distance(double, double, double, double, double, double): double
+ distance(double, double, double, double, char): double
+ err(String): void
+ getCoordsOfBlock(int, int, int, int): Coordinates
+ getFolderSize(File): long
+ getHeightWithRotation(double, double): int
+ getPixelSize(int): double
+ getPixelsX(double, int, int): int
+ getPixelsX(double, int, int, int): int
+ getPixelsY(double, int, int): int
+ getPixelsY(double, int, int, int): int
+ getRofRotatedText(int, int, int, int, int): Rectangle
+ getStringH(Font, String): int
+ getStringW(Font, String): int
+ getTileCoords(double, double, int, int): Coordinates
+ getUsedMemory(): long
+ getWidthWithRotation(double, double): int
+ getZoom(): int
+ gpsSmooth(ArrayList<Coordinates>): void
+ gpsToMapX(double): int
+ gpsToMapX(double, int): int
+ gpsToMapY(double): int
+ gpsToMapY(double, int): int
+ isPointVisible(Coordinates, boolean, double, double): boolean
+ lgisGetPixelsX(double, int, int): int
+ lgisGetPixelsY(double, int, int): int
+ lgisLatLonToXY(double, double): Coordinates
+ lgisPixelsToLat(int, int, MapTile): double
+ lgisPixelsToLon(int, int, MapTile): double
+ lgisTileE(int, int): double
+ lgisTileN(int, int): double
+ log(String): void
+ log(String, long): void
+ parseInt(String): int
+ pixelsToLat(int, int, int): double
+ pixelsToLat(int, MapTile, boolean): double
+ pixelsToLon(int, int, int): double
+ pixelsToLon(int, MapTile): double
+ print(String): void
- rad2deg(double): double
+ setZoom(int): void
+ sleep(long): void
+ test(double, double): int
+ tileGpxOsm(int, int): String
+ Tools()
+ Tools(int)
+ translateTextY(int, int, int): int
+ translateZoomLgis(int, int, int, int, int, int): Coordinates
+ xToLon(int): double
+ xToLon(int, int): double
+ XYToLatLon(int, int): Coordinates
+ yToLat(int): double
+ yToLat(int, int): double
«property get»
+ getmLat(): double
+ getmLon(): double
Coordinates
+ addDistance(double, double): void
+ addDistance(Coordinates, double, double): Coordinates
+ Coordinates(double, double)
+ Coordinates(double, double, long)
+ Coordinates(double, double, int, int)
+ Coordinates(double, double, int, Date)
+ Coordinates(int, int)
+ Coordinates(int, int, int, int)
+ getDate(): Date
+ getElevation(): int
+ getLat(): double
+ getLon(): double
+ getOfX(): int
+ getOfY(): int
+ getOsmID(): long
+ getStringCoords(): String
+ getValue(): long
+ getX(): int
+ getY(): int
+ isEdit(): boolean
+ isUsed(): boolean
+ setDate(Date): void
+ setEdit(boolean): void
+ setElevation(int): void
+ setLat(double): void
+ setLon(double): void
+ setNodeId(long): void
+ setOfX(int): void
+ setOfY(int): void
+ setOsmID(long): void
+ setUsed(boolean): void
+ setValue(long): void
+ setX(int): void
+ setY(int): void
+ toString(): String
CoordinateInfo
+ CoordinateInfo(double, double, String)
+ getInfo(): String
+ setInfo(String): void
CoordinatesParser
+ filter(ArrayList<String>, boolean): void
+ isValidCoordinate(double): boolean
+ parseCoordinates3(String, boolean, boolean): ArrayList<Coordinates>
+ parseCoordInfo(ArrayList<String>, ArrayList<Coordinates>, int, Coordinates): int
+ parseDCoordinate(String): double
+ parseDegreeMinuteCoordinate(String): double
+ parseDMSCoordinate(String): double
+ parseLatLon(String): double
+ parseLineValue(String): long
+ split(String): ArrayList<String>
+ splitAndAdd(String, String, ArrayList<String>): void
+ splitLine(String): ArrayList<String>
DrawTools
+ addAlert(String, AlertType): void
+ brighten(Color, double): Color
+ draw(Graphics2D, int, int): void
+ drawAlerts(Graphics2D): void
~ DrawTools()
+ getInstance(): DrawTools
+ getLoading(): int
+ imgToGrey(BufferedImage): BufferedImage
+ setLoading(int): void
+ startLoading(): void
+ stopLoading(): void
FileTools
+ addMap(String, String, String): void
+ findOsmFile(File): File
+ findUJFile(File): File
+ folderSize(File): long
+ listCache(String, ArrayList<File>): void
+ loadDoc(String): org.w3c.dom.Document
+ loadMainLayers(): ArrayList<Layer>
+ loadMaps(String): ArrayList<Layer>
+ loadOsm(String): ArrayList<Coordinates>
+ loadSecondaryLayers(): ArrayList<Layer>
+ saveDoc(String, org.w3c.dom.Document): void
+ setNameTag(String, String): void


class rightclick
RCAddPoint
- manager: PointsManager
+ click(Rectangle, Coordinates, int, int): void
+ RCAddPoint(PointsManager)
«interface»
RightClick
+ rigthClick(Rectangle): ArrayList<RCMenuItem>
RCCopyCoords
+ click(Rectangle, Coordinates, int, int): void
+ RCCopyCoords()
RCPoint
+ click(Rectangle, Coordinates, int, int): void
+ RCPoint(Coordinates)
RCAnimation
- anim: AnimItem
+ click(Rectangle, Coordinates, int, int): void
+ RCAnimation(AnimItem)
RightClickMenu
- clearItems: boolean = false
- core: Animator
- items: ArrayList<RCMenuItem> = new ArrayList<R...
- position: Rectangle
- regObjects: ArrayList<RightClick> = new ArrayList<R...
- tempItems: ArrayList<RCMenuItem> = new ArrayList<R...
- visible: boolean = false
+ click(MouseEvent, Rectangle): void
+ hover(Rectangle): void
+ register(RightClick): void
+ registerItem(RCMenuItem): void
+ render(Graphics2D): void
+ RightClickMenu(Animator)
RCMenuItem
backgroundColor: Color = Color.white
f: Font = new Font("Arial...
foregroundColor: Color = Color.black
height: int = 20
hover: boolean = false
hoverColor: Color = new Color(25,25...
img: BufferedImage
text: String
width: int = 200
+ click(Rectangle, Coordinates, int, int): void
+ draw(Graphics2D, int, int): void
+ drawBackGround(Graphics2D, int, int): void
+ drawHover(Graphics2D, int, int): void
+ drawIcon(Graphics2D, int, int): void
+ drawText(Graphics2D, int, int): void
+ getBackgroundColor(): Color
+ getF(): Font
+ getForegroundColor(): Color
+ getHeight(): int
+ getHoverColor(): Color
+ getImg(): BufferedImage
+ getText(): String
+ getWidth(): int
+ hover(Rectangle, int, int): void
+ intersects(Rectangle, int, int): boolean
+ isHover(): boolean
+ RCMenuItem()
+ RCMenuItem(BufferedImage, String)
+ RCMenuItem(BufferedImage, String, Color, Color, Color)
+ setBackgroundColor(Color): void
+ setF(Font): void
+ setForegroundColor(Color): void
+ setHeight(int): void
+ setHover(boolean): void
+ setHoverColor(Color): void
+ setImg(BufferedImage): void
+ setText(String): void
+ setWidth(int): void